

The Tabletop Rover Light Show

Six lamps drive an imaginary rover through a tiny adventure

The rover is light, logic, and imagination—no motors required.

Lesson	021 — project	Build time	about 35 minutes
Board	Arduino Mega 2560	Power	USB only
Visible result	route, turn, obstacle, finish	Finish	automatic at 12 seconds

1 The game

Build two banks of three lamps. Each bank is an imaginary wheel: **forward**, **reverse**, and **enable**. Together the six lamps replay a route generated by the real `RoverController`, `UltrasonicRanger`, and two `MotorIntent` objects.

The show starts by itself, drives forward, turns, freezes for a pretend obstacle, continues the route, and ends dark. A 100 ms blackout prevents an instant direction reversal. At 12 seconds the sketch shuts down every output, even if the route or synthetic observations were changed.

This is the complete supported circuit. Use only the Mega, six ordinary LEDs, six 1 k Ω resistors, a breadboard, and jumpers. Do not connect a motor, H-bridge, motor supply, relay, wheel, or battery. A kit family name does not identify their electrical limits.

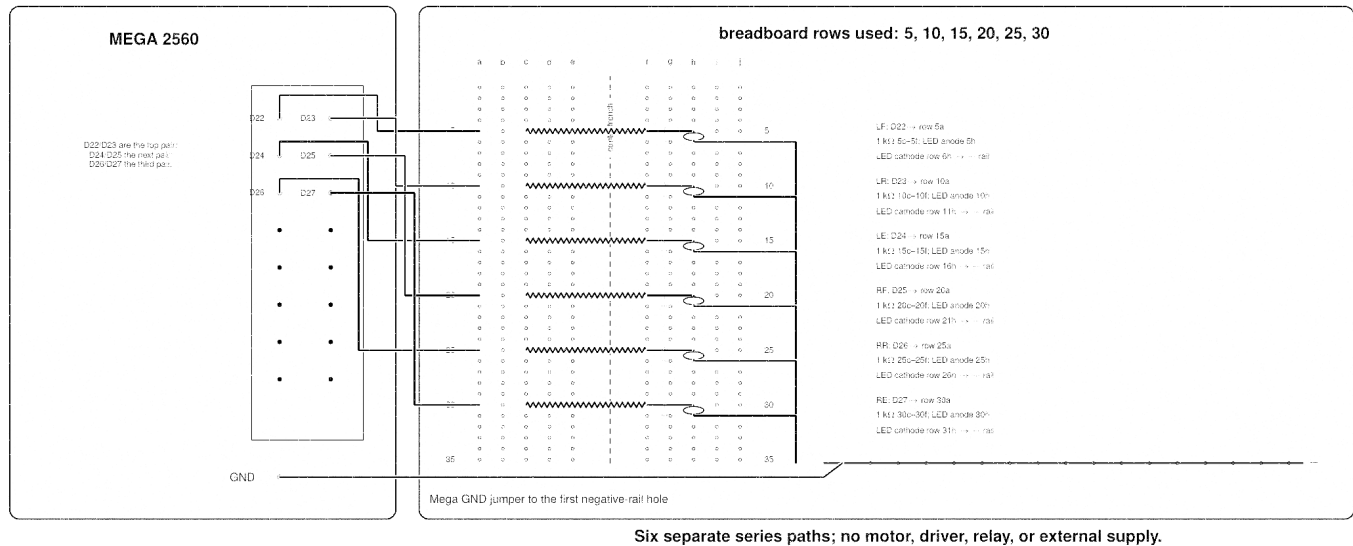
2 Meet the six characters

Lamp	Mega pin	Meaning
LF	D22	left side requests forward
LR	D23	left side requests reverse
LE	D24	left side has nonzero applied duty
RF	D25	right side requests forward
RR	D26	right side requests reverse
RE	D27	right side has nonzero applied duty

The built-in D13 lamp turns on only after every object initializes. It is the “cast is ready” signal; it is not one of the six route lamps.

3 Wire the literal layout

The complete USB-only rover simulator



The paired Mega header and breadboard coordinates are literal. Every output has its own resistor and LED; only the cathode returns join at the negative rail. The first hole of that rail returns to Mega GND.

Signal	Complete path
LF	D22 to 5a; 1 k Ω from 5c to 5f; LED long leg at 5h, short leg at 6h; 6j to negative rail
LR	D23 to 10a; resistor 10c–10f; long leg 10h, short leg 11h; 11j to negative rail
LE	D24 to 15a; resistor 15c–15f; long leg 15h, short leg 16h; 16j to negative rail
RF	D25 to 20a; resistor 20c–20f; long leg 20h, short leg 21h; 21j to negative rail
RR	D26 to 25a; resistor 25c–25f; long leg 25h, short leg 26h; 26j to negative rail
RE	D27 to 30a; resistor 30c–30f; long leg 30h, short leg 31h; 31j to negative rail
Return	first negative-rail hole to a labeled Mega GND pin

Unplug USB before moving a wire. The LED long leg is the anode. If the body has a flat edge, that edge usually marks the short cathode leg; check the actual LED before placing it.

4 Build in three quick wins

Win 1—one imaginary wheel. Build LF, LR, and LE on rows 5, 10, and 15. Upload the sketch. You should see left-side forward intent and later the turn. If these three work, the left half of the story is already visible.

Win 2—the rover appears. Add RF, RR, and RE on rows 20, 25, and 30. Both sides now drive forward together. During the turn the direction banks disagree on purpose: that is how a two-wheel rover pivots.

Win 3—the obstacle trick. Watch near 2.5 seconds. Both enable lamps go dark for a full second because the synthetic range drops from 500 mm to 150 mm. Nothing is pressed and no sensor is wired; the observation is a fixed, replayable part of the sketch. At 3.5 seconds the clear range returns.

Each payoff depends on the earlier wiring. A missing resistor path, reversed LED, shared row, or wrong Mega pin removes the matching character from the story instead of creating a separate test chore.

5 Read the twelve-second story

Moment	What the six lamps show	What is happening
startup	six dark; then D13 on	objects acquired before route intent
first act	LF, LE, RF, RE	both imaginary wheels forward
near 2.5 s	LE and RE dark	synthetic obstacle holds both sides stopped
after 3.5 s	route lamps resume	fresh clear range releases the hold
turn	opposite direction patterns, with a brief all-dark gap	requested reversal waits for <code>MotorIntent</code> dead time
finish	all six dark	route ended with stopped intent
12 s	D13 and all six dark	fixed automatic shutdown released outputs

Small timing differences in LED brightness are normal because the enable signals use duty values. The ordering is deterministic: identical timestamps and observations produce identical controller snapshots.

```
void loop ()
{
  observeSimulatedRange  (nowUs);
  observeSimulatedWheels (now);
  decideRoverMotion      (now);
  actuateMotorEvidence    (now);
}
```

The canonical loop keeps the story in dependency order: observe, decide, actuate. Serial is unnecessary; the six lamps are the primary result.

6 Change the adventure

1. Change the near distance from 150 mm to 100 mm. Predict why the visible story stays the same: both values remain inside the stop threshold.
2. Move the obstacle window from 2.5–3.5 seconds to 1.5–2.5 seconds. The blackout moves earlier without changing a wire.
3. Change the first route step from eight edges to four. The turn should arrive earlier, proving the route data drives the story.
4. Swap the physical LF and RF LEDs. Decide which facts remain true and which labels become misleading.

7 If a character disappears

What you see	Inspect with USB unplugged
Exactly one lamp never lights	its numbered Mega pin, resistor row, LED polarity, and cathode jumper
Three left or three right lamps fail	the corresponding D22–D24 or D25–D27 header positions
Several unrelated lamps copy each other	accidental shared breadboard rows or signal jumpers touching
Directions work but one enable stays dark	D24 or D27 and its complete series path
No obstacle blackout	confirm the Lesson 021 sketch was uploaded, not Lesson 020
Any lamp remains after 12 seconds	reset once; if repeated, unplug USB

8 What the repository checks behind the scenes

Automated tests replay every route action, obstacle hysteresis boundaries, stop dominance, stale or invalid range, wheel faults, route timeouts, rollover, restart, shutdown, and byte-identical traces. Arduino compilation and size checks cover the Mega sketch. Those checks protect the project without becoming learner paperwork.

9 Why there are no real wheels

The software emits *intent*; LEDs make that intent visible. Real motion would require an exactly identified driver and motor, primary datasheets, measured stall current, a separate current-limited load supply, flyback and thermal design, a rated physical power disconnect, guarding, and physical evidence. None is authorized by resemblance to a retail listing. This lesson therefore ends at the safer and more useful USB-only simulator.

Status: host verified; no physical observation is claimed. The HTML lesson, canonical sketch, and automated tests remain the current reference.